

Laoh

Your portfolio bought AI. *The P&L can't see it.*

A capability diagnostic for the engineering teams inside your portcos - who can actually use AI, what's blocking them, what to fix first.

From the team that screens forward-deployed engineers for a frontier AI lab, a vertical AI company, and 3 YC-backed AI cos.

AI impact has moved from story to diligence requirement.



AI license spend

up and to the right since 2023 - copilot seats, agent tools, pilots

EBITDA impact

the question everyone is asking - in every board meeting, and now in every sale process

Engineering teams sit at five distinct levels of AI capability.

	WHAT IT IS	WHAT YOU'D SEE THEM DOING	WHAT COMES OUT
L1 chat-window operator	AI as a better search box	pastes questions into a chat window; no repo context; accepts answers on vibes	snippets; the same work, slightly faster
L2 agent-native, context-naive	agent tools driven like a chat window <i>the trap - the tools look advanced, so it feels like mastery; most self-described AI-native engineers are here</i>	one-shot prompts in Cursor or Claude Code; no instructions file; doesn't read the diff	unreviewed output piles up; feels fast, ships slop
L3 context engineer	supplies context on purpose - the dividing line	keeps a persistent instructions file; picks the right files, conventions and constraints; writes a spec before prompting; reviews the output	the agent starts right; real gains begin
L4 high-context + verified	rich context plus a hard oracle	adds screenshots, diagrams and worked examples instead of prose; writes tests first so passing is proof; reads every diff	verified changes at speed; quality stops depending on judgment calls
L5 fan-out + refusal	parallel agents, deliberate limits	decomposes work and fans it out to parallel agents - safe because verification is automated; has found where AI is negative and stopped using it there	several verified workstreams per person; judgment becomes the output

Every level is claimed in interviews. Each one is verifiable only by watching the work - which is the diagnostic.

A team at L5 ships several times the output of the same team at L2.

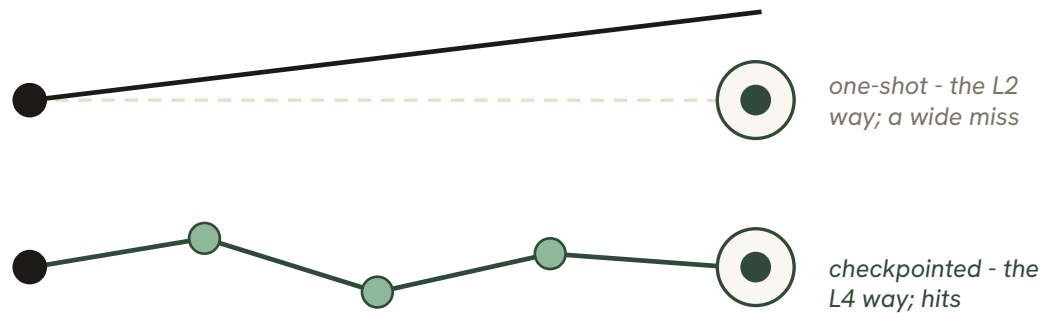
	WHAT GOES IN	WHAT HAPPENS	WHAT COMES OUT
L2	one-shot prompts into an agent tool	no context supplied, no spec, diffs land unread	feels fast; slop and rework eat the gain
L3	a spec and deliberately chosen context	the agent starts right; the person reviews every output	the dividing line - real gains begin, one stream at a time
L5	decomposed tasks fanned out to parallel agents	tests-first verification gates every change; AI kept off tasks where it's negative	several verified workstreams per person; cycle time collapses

Why the order matters. L5 fan-out is only safe because L4 verification is automated - parallel agents without tests-first just multiply unreviewed diffs.

Capacity per head and cycle time are the lines a value-creation plan is priced on.

Two disciplines move a team up the ladder - checkpoints and context engineering.

CHECKPOINT-DRIVEN DEVELOPMENT



Plan meticulously. Read what the AI built at every gate, correct, move. Feels slower - ships several times faster, without the slop.

CONTEXT ENGINEERING



context supplied on purpose, every time - the L3 dividing line

A dumped context degrades the model with every turn. Choosing it deliberately - conventions, constraints, the files that matter - is the L3 dividing line. Screenshots, worked examples and tests-first make it L4.

Simple to learn, real only when applied. Like sales or leadership, courses alone get you a tenth of the way - these skills are built inside the live workflow, or not at all.

And skip the hype. "Loop engineering", "Ralph loops", fully autonomous everything - influencer content, counterproductive for enterprise production code. Engineers inside frontier labs don't build this way.

Value per token rises with every level your people climb.

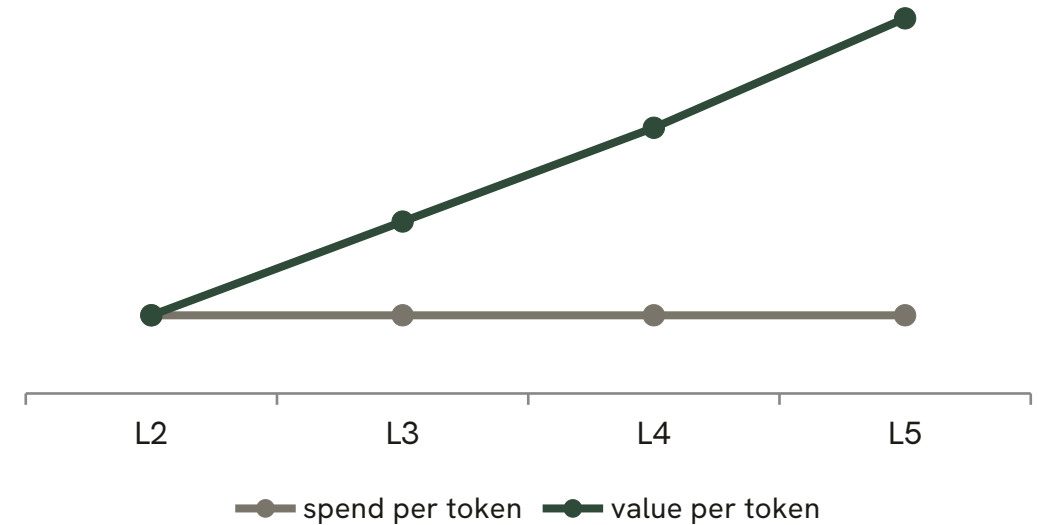
WHAT WE WATCH

Where each person applied the disciplines - and where they didn't. Task by task: did a plan and checkpoints exist, was the work reviewed at each gate, was context engineered or dumped.

WHAT THAT SHOWS

Exactly where the productivity gap sits, person by person - the tasks where a discipline should have been applied and wasn't. That visibility alone shows where value per token is leaking.

ILLUSTRATIVE



So we hand you exactly that - a scorecard for every person, and the map for every team.

Developers misjudge their own AI productivity by about 40 points.

+20%

how much faster developers believed AI made them

-19%

how much slower they actually were

Randomized trial, 246 real tasks (METR, 2025)

Every adoption survey in your portfolio carries this same gap. Placing people on the ladder requires watching them work.

We measure capability by observing days of real work.

I

Observability agents sit in the coding sessions

Coding sessions only - never the rest of the laptop. Announced, approved, zero engineer hours lost.

2

They record how work happens

Engineering leads decide what gets measured for each role - which tasks go to agents, whether checkpoints and engineered context were applied, how results get verified.

3

We score each person against the level definitions

A level - plus the observed evidence behind it.

THE CALIBRATION

A role-matched live simulation calibrates every score.

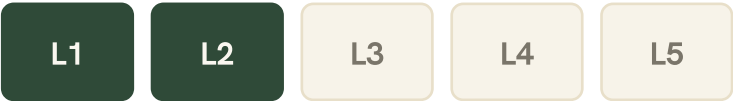
The same instrument we use to screen forward-deployed engineers for frontier labs.

Workflow signals only - code never leaves the portco's environment · reports go to the person and the CEO · one 30-min security review with the portco CTO is the full approval

Every person's level comes with the evidence behind it.

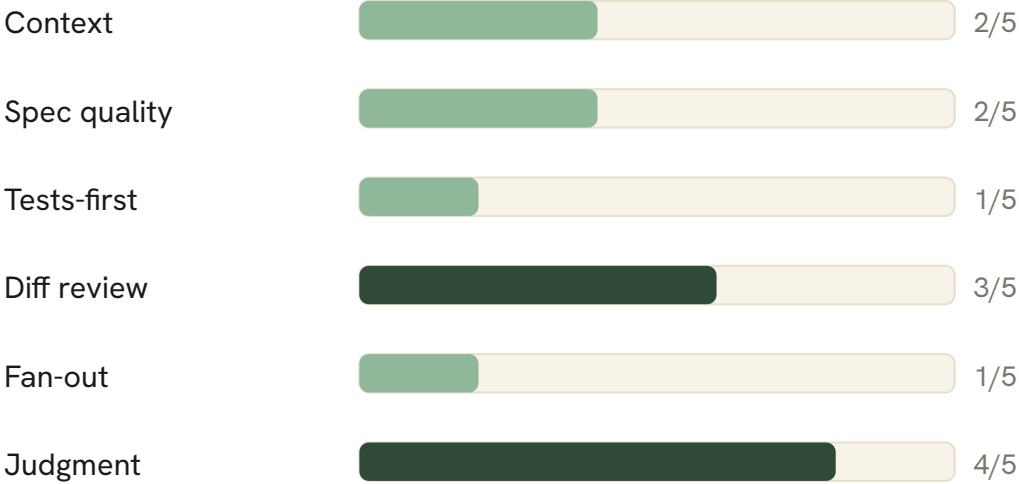
A. Mehta

Integrations engineer · Portco A · assessed 12 Jun



operates at L2 - agent-native, context-naive; drives the tool like a chat window

SCORED DIMENSIONS



WHAT WE SAW

“one-shot prompt, then merged the diff unread”

“ran the code once and called it verified”

“knew exactly when not to use the agent - strongest signal”

path to L3: build the instructions file · supply files on purpose · write the spec before prompting

A growth map, never a report card.

Three moves take a portco from baseline to proven capability gains.

I

Measure

2-3 weeks, one portco: capability map, readiness scorecard, sequenced 100-day plan.

2

Move the levels

12-week cohorts by assessed level - skills like checkpoints and context engineering, drilled on the team's real tickets inside the live workflow.

3

Prove it moved

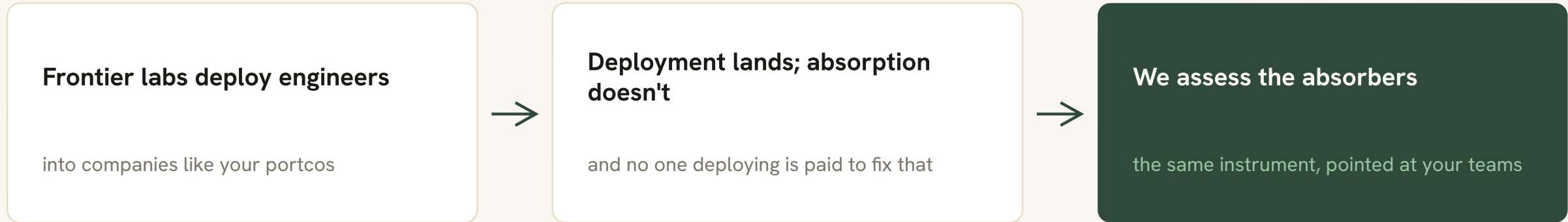
Quarterly re-measure with telemetry on - movement in numbers your IC can read.

ILLUSTRATIVE

Only what's measured improves: hand-corrected agent output ↓ + agent-authored share ↑ → cycle time on comparable tickets ↓ → capacity per head ↑

An ugly baseline seen now is pre-diligence on your terms.

We've watched AI deployment fail from the other side of the table.



We screen FDEs for **a frontier AI lab · a vertical AI company · 3 YC-backed AI cos**

Pulkit Walia

Built the engineering assessment and skilling system at Urban Company (\$3B IPO), a tech-enabled trades marketplace.

Rohit Jain

25,000+ engineers trained. Hundreds of big-tech interview loops run from the other side of the table.

START HERE

Start with one portco - \$1,000 a seat, results in three weeks.

WHAT YOU GET

A level scorecard for every person

with the observed evidence behind it

The team map by function

where the whole portco sits, L1-L5

The leak map

where value per token is being lost, task by task

A sequenced 100-day plan

what to fix first - people or infrastructure

TYPICAL RUN 30-40 seats · \$30-40k · fund or portco budget

THEN, IF IT EARNS IT

one portco

portfolio dashboard

upskilling

Start where engineering cost is the thesis.

THE LINE TO FORWARD TO YOUR IC

“For \$40k and three weeks we get a measured answer on whether [portco]’s AI spend is building capability or shelfware - before a buyer’s diligence team asks.”

30 mins to scope it. We'll bring the instrument. Pulkit Walia · laoh.ai

Person scorecards roll up into a team map by function.



SAMPLE PORTCO **The read:** most engineers sit at L2 - agent tools adopted, context discipline missing. No one past L3. The 100-day plan starts at the L3 dividing line, backend first.